NEXUSRESEARCH INTERVIEW MASTERY

Comprehensive Technical Interview Question Bank & Defense Guide

A Complete Technical Question Bank (Beginner to Extreme) Covering Architecture, Full-Stack Implementation, Research Engine, LLM Grounding, Database, Security, and System Design

[SOURCE OF TRUTH GOVERNANCE] This question bank is built directly from the NexusResearch codebase. It prepares candidates to defend every architectural choice, API contract, database schema, search provider, evidence grounding step, and deployment mechanism. Every answer connects to actual code, real test metrics, and production-tested patterns.

Project: NexusResearch Enterprise Platform
Technology Stack: Next.js 16 (React 19, TS 5), FastAPI (Python 3.10+), SQLAlchemy 2.0, PostgreSQL/SQLite, Gemini 2.5, python-docx, reportlab, NextAuth v5
Question Bank Scope: 39 Structured Sections (250+ In-Depth Technical Questions & Case Studies)
Publication Date: August 24, 2026
Repository Source: https://github.com/TejeshwarDivekar/NexusAI.git

1. Project Introduction & Value Proposition

Q1.1: Tell me about your project in detail.
NexusResearch is a full-stack, enterprise-grade AI research assistant engineered for verifiable literature inquiry, multi-registry academic retrieval, interactive evidence exploration, and publication-ready IEEE Word (.docx) and academic PDF (.pdf) document generation. It eliminates hallucinations by enforcing a strict 7-stage pipeline querying OpenAlex, arXiv, PubMed, Europe PMC, and Crossref.

Q1.2: What makes it different from existing AI assistants?
Standard AI chatbots hallucinate citations and summarize without verifiable proof. NexusResearch performs live concurrent searches on authoritative registries, extracts sentence-level evidence quotes, audits empirical conflicts, and validates citations with a deterministic CitationValidator.

2. System Architecture & Request Lifecycle

Q2.1: Trace a query from submission to final response:
1. Query submitted in Next.js frontend (`ResearchComposer.tsx`).
2. Proxied via `src/app/api/v1/[...path]/route.ts` to FastAPI backend.
3. `ResearchEngine` classifies inquiry, queries 7 registries concurrently via `asyncio.gather` with timeouts.
4. `SourceRelevanceScorer` filters low-relevance candidates.
5. `EvidenceService` extracts quotes; `ContradictionService` audits discrepancies.
6. `GeminiProvider` synthesizes report with citations [1], [2].
7. `IEEEDocumentGenerator` and `AcademicPDFGenerator` compile files.
8. SQLAlchemy saves session to PostgreSQL/SQLite; UI displays report and Evidence Matrix.

3. 7-Stage Multi-Registry Research Engine

- **Stage 1: Query Classification & Topic Cleaning:** Generates formal title and sub-queries.
- **Stage 2: Multi-Source Real Data Retrieval:** Queries OpenAlex, arXiv, PubMed, Europe PMC, Crossref, Wikipedia, DuckDuckGo.
- **Stage 3: Hard Relevance Gating:** Discards low-scoring matches (<0.45 relevance score).
- **Stage 4: Verified Evidence Extraction:** Extracts sentence-level quote excerpts.
- **Stage 5: Contradiction Auditing:** Detects empirical conflicts and differing conclusions.
- **Stage 6: Quantitative Analysis:** Extracts percentages, metrics, and tabular data.
- **Stage 7: Answer-First LLM Synthesis:** Synthesizes report constrained to retrieved quotes.

4. Interactive Evidence Matrix & State Restoration

Q4.1: How does the Evidence Matrix work?
When a user clicks an evidence card in the right panel, the center panel transitions to `EvidenceDetailView` displaying the paper quote, metadata, and on-demand AI explanation. Clicking '← Back to Answer' restores the research report and scroll position instantly without re-running the research query.

5. Top 10 Must-Master Interview Questions

1. **Why Next.js + FastAPI?** Separation of client rendering and high-concurrency async Python pipeline.
2. **How do you prevent hallucinations?** Strict prompt grounding, quote extraction, and post-synthesis CitationValidator.
3. **Why SQLite in dev and PostgreSQL in prod?** Zero-config local development vs robust multi-tenant cloud persistence.
4. **How does OAuth sync work?** NextAuth handles Google handshake; `/api/v1/auth/oauth_sync` creates backend user and returns JWT.
5. **How is IEEE Word formatting generated?** `python-docx` builds double-column XML layout with structured sections.
6. **How do you handle API timeouts?** `httpx.AsyncClient(timeout=10.0)` with `asyncio.gather(..., return_exceptions=True)`.

7. **How is multi-tenancy enforced?** Every SQL query scopes by `user_id` at the database level.
8. **How do you test the application?** 30 automated Pytest tests in `backend/tests/` and Next.js production build checks.
9. **What was a real production bug you fixed?** Added `reportlab` to requirements and created `run.py` for integer PORT parsing on Railway.
10. **What are future improvements?** Native pgvector embeddings search, APA/MLA export formats, and asynchronous Celery queues.

[COMPLETE QUESTION BANK AVAILABLE] For the full 39-chapter comprehensive question bank with 250+ technical questions, code walkthroughs, and mock interviews, see the companion document: NexusResearch_Interview_Question_Bank.docx.